Rapport de Travaux Pratiques

TP 7.1 à 7.4 : Sérialisation, Validation et Sécurité

Séance 7 — Applications Réparties et Cybersécurité

Étudiant : [**Votre Nom**]

1ère année cycle ingénieur cybersécurité

25 avril 2026

Table des matières

1 TP 7.1 — Contrat JSON + Validation Stricte **2**
1.1 Objectif 2
1.2 Tableau du contrat des données 2
1.3 Implémentation Python (`tp7_1_validation.py`) 2

2 TP 7.2 — Versioning JSON (Compatibilité) **4**
2.1 Objectif 4
2.2 Matrice de compatibilité 4
2.3 Implémentation Python (`tp7_2_versioning.py`) 4

3 TP 7.3 — Protobuf en Python **5**
3.1 Objectif 5
3.2 Schéma (`document.proto`) 5
3.3 Comparaison JSON vs Protobuf (`tp7_3_protobuf.py`) 5

4 TP 7.4 — Politique "Safe Serialization" **6**
4.1 Objectif 6
4.2 Tableau de politique d'usage 6
4.3 Checklist "Secure Deserialization Baseline" 6

1 TP 7.1 — Contrat JSON + Validation Stricte

1.1 Objectif

Définir un contrat JSON formel pour les entités `Document` et `UserPublic`, puis implémenter la sérialisation, la désérialisation et la validation stricte en Python (approche *fail closed*).

1.2 Tableau du contrat des données

Champ	Type	Obligatoire	Règles de validation	Exemple
`id`	int	Oui	Entier > 0	42
`title`	str	Oui	1–200 caractères (non vide après strip)	"Rapport Q1"
`author`	str	Oui	1–100 caractères	"Alice"
`tags`	list[str]	Non	Max 20 éléments, 1–50 chars par tag	["finance"]
`classification`	str	Non	Allowlist (public, internal, confidential, secret)	"confidential"
`created_at`	str	Non	Format ISO 8601 UTC (fini par Z)	"2026-01-15T10 :30 :00Z"
`username`	str	Oui	3–30 caractères, alphanumérique + underscore	"alice_d"
`display_name`	str	Oui	1–100 caractères	"Alice Dupont"
`role`	str	Oui	Allowlist (viewer, editor, admin)	"editor"

Table 1 – Contrat JSON pour `Document` et `UserPublic`

1.3 Implémentation Python (`tp7_1_validation.py`)

```python
def deserialize_document(raw: str) -> Document:
    try:
        data = json.loads(raw)
    except json.JSONDecodeError:
        raise ValueError("Payload invalide")

    errors = []

    # 1. Validation de la presence et des types
    if "id" not in data or not isinstance(data["id"], int) or data["id"] <= 0:
        errors.append("'id' doit etre un entier positif > 0")

    if "title" not in data or not isinstance(data["title"], str):
        errors.append("Champ 'title' invalide")
    elif len(data["title"].strip()) == 0 or len(data["title"].strip()) > 200:
        errors.append("Taille de 'title' non respectee")

    # 2. Validation par allowlist
    classification = data.get("classification", "internal")
    if classification not in ALLOWED_CLASSIFICATIONS:
        errors.append("Classification non autorisee")

    # 3. Fail closed : tout defaut declenche le rejet total
    if errors:
```

```
        logger.warning("Validation Document echouee : %s", errors)
        raise ValueError("Payload invalide")

    return Document(id=data["id"], title=data["title"].strip(),
                    author=data["author"], tags=data.get("tags", []),
                    classification=classification)
```

Listing 1 – Extrait de la fonction de désérialisation avec validation

Analyse Sécurité

L'approche **fail closed** garantit qu'aucune donnée partiellement valide n'atteint la couche métier. De plus, l'exclusion explicite des champs privés (`_internal_note`, `_password_hash`) lors de la sérialisation empêche toute fuite de données vers le client (Data Exposure).

2 TP 7.2 — Versioning JSON (Compatibilité)

2.1 Objectif

Faire évoluer le contrat du document d'une **v1** vers une **v2** en ajoutant les champs `tags` et `classification`, tout en maintenant la rétrocompatibilité et en traitant les champs inconnus.

2.2 Matrice de compatibilité

Payload	Lecteur	Accepté ?	Valeurs appliquées	Justification
V1	V2	**Oui**	`tags`=[], `class`="internal"	Les champs V2 sont optionnels avec valeurs par défaut.
V2	V2	**Oui**	Valeurs du payload	Correspondance exacte au contrat.
V2 altéré	V2	**Non**	*Rejeté*	La classification hors *allow-list* est rejetée (Fail Closed).
V2+inconnu	V2	**Oui**	Champs inconnus ignorés	Tolérance au changement (Robustness Principle). Empêche le *Mass Assignment*.

Table 2 – Matrice d'évolution des contrats JSON

2.3 Implémentation Python (`tp7_2_versioning.py`)

```python
def deserialize_document_v2(raw: str) -> dict:
    data = json.loads(raw)

    # 1. Verification des champs obligatoires (V1)
    for f in ("id", "title", "author"):
        if f not in data:
            raise ValueError(f"Champ manquant : {f}")

    # 2. Champs optionnels avec defaults (V2)
    tags = data.get("tags", [])
    classification = data.get("classification", "internal")

    # 3. Restitution stricte : les champs inconnus sont abandonnes
    return {
        "id": data["id"],
        "title": data["title"],
        "author": data["author"],
        "tags": tags,
        "classification": classification
    }
```

Listing 2 – Désérialiseur tolérant (Forward Compatibility)

3 TP 7.3 — Protobuf en Python

3.1 Objectif

Utiliser **Protocol Buffers** pour sérialiser le document, évaluer le gain en taille, et comprendre le mécanisme de versioning natif.

3.2 Schéma (`document.proto`)

```
syntax = "proto3";
package demo;

message Document {
    int32 id = 1;
    string title = 2;
    string author = 3;
    repeated string tags = 4;
    string classification = 5;

    // Ajout d'un champ optionnel (v2)
    int32 priority = 6;
}
```

3.3 Comparaison JSON vs Protobuf (`tp7_3_protobuf.py`)

Critère	JSON	Protobuf	Commentaire
Taille payload	∼150 octets	∼50 octets	**Protobuf 3x plus compact**
Lisibilité	Humaine	Binaire (Illisible)	Utile pour debug vs perf
Validation	Applicative (Manuelle)	Intrinsèque (Schéma)	Protobuf garantit les types
Tooling	Aucun (stdlib)	`protoc` requis	Étape de compilation
Compatibilité	Par convention	Native (Numéros)	Protobuf gère les UnknownFields

Table 3 – Comparatif des formats d'échange

Analyse : Compatibilité Protobuf

En Protobuf, les champs sont identifiés par leur **numéro** et non par leur nom textuel. Si un lecteur *V1* reçoit un message *V2* avec le champ `priority = 6`, il le considérera comme un **Unknown Field**. Il le stockera en mémoire sans le parser et, de manière transparente, s'il doit le resérialiser plus tard, les octets inconnus seront préservés. C'est un avantage majeur pour la *forward compatibility*.

4 TP 7.4 — Politique "Safe Serialization"

4.1 Objectif

Définir une politique de sécurité formelle sur l'usage de la sérialisation, en identifiant quand l'usage de formats exécutables (comme `pickle`) est strictement interdit.

4.2 Tableau de politique d'usage

Source des données	Format	Contrôles Requis / Justification
API REST (Externe)	JSON	Entrée non fiable. **Validation stricte**, limitation de taille, authentification. **Pickle INTERDIT**.
Inter-services (Interne)	Protobuf	Format inerte et rapide. Requiert **mTLS** ou **HMAC** car le réseau interne reste une zone de risque (Zero Trust).
Cache local (Même app)	`pickle`	**Autorisé** uniquement si la donnée ne provient pas du réseau et n'est pas modifiable par l'utilisateur.
Fichier uploadé	JSON/XML	Toujours utiliser `yaml.safe_load()` ou `defusedxml`. Limiter la taille avant parsing.

Table 4 – Politique de formats de sérialisation par niveau de confiance

4.3 Checklist "Secure Deserialization Baseline"

- [] **Interdire `pickle`**, `marshal`, ou `yaml.load()` pour toutes les données traversant le réseau. Utiliser des formats **inertes** (JSON, Protobuf).
- [] **Limiter la taille** (Content-Length) avant même d'invoquer le parseur pour éviter les attaques DoS (ex : Bombes XML ou gros JSON).
- [] **Validation exhaustive** : valider la présence, les types, et les valeurs (allowlists) juste après la désérialisation.
- [] En cas de flux inter-services asynchrones (Kafka/RabbitMQ), **signer les payloads** (HMAC) pour garantir l'intégrité de la donnée.
- [] Utiliser le pattern **Fail Closed** : rejeter toute donnée inattendue sans exposer d'informations système dans le message de retour.